Project Overview — Simple Mail Server & Client

1. Architecture Description

The project implements a simple text-based mail system using a client–server model over TCP sockets.
The communication follows a minimal custom line-based protocol inspired by SMTP-like semantics.

Server

- The server is a single-threaded, blocking TCP server written in modern C++ (C++17).
- It listens for incoming connections on a configurable port (`6543` by default).
- For each connected client, it enters a command–response loop, interpreting the following commands:
  - `SEND` – store a new mail for a user
  - `LIST` – list all stored messages for a user
  - `READ` – retrieve the full content of a specific message
  - `DEL` – delete a specific message
  - `QUIT` – close the connection
- Mails are stored as plain text files under a local `mail-spool/<username>/` directory, with filenames `<id>.txt`.
- Each message file contains simple headers:

```
FROM: <sender>
SUBJECT: <subject>

<message body>
```

- The server performs input validation and uses helper functions (`is_valid_username`, `is_valid_subject`, `parse_message_id`) to ensure protocol integrity.
- Responses are simple control lines like `OK\n` or `ERR\n`, keeping the protocol human-readable.

Client

- The client is a console-based interactive program, also implemented in C++.
- It connects to the server via TCP, sends commands, and displays responses.
- It supports full interaction:
  - Prompts for sender, receiver, subject, and message body for `SEND`.
  - Displays message lists (`LIST`) and contents (`READ`).

    - Allows deleting mails (`DEL`) and exiting (`QUIT`).
- Communication is line-oriented (CRLF terminated) to maintain cross-platform protocol compatibility.

2. Used Technologies

Category	Technology	Purpose
Language	C++17	Core implementation for both client and server
Standard Libraries	`<filesystem>`, `<fstream>`, `<sstream>`, `<thread>`, `<sys/socket.h>`, `<netinet/in.h>`	File management and socket I/O
Build System	Make or direct `g++` compilation	Simple and portable compilation
Platform	POSIX / Linux	Uses standard Unix socket APIs

3. Development Strategy and Adaptations

Development Strategy

The project was built in incremental, test-driven stages:

1. Base server setup: implemented socket creation, binding, listening, and connection handling.
2. Core command `SEND`: introduced message writing and storage using `<filesystem>`.
3. Protocol helpers: extracted common validation and socket reading logic into modular helper files (`readLineFromSocket.cpp`, `protocolValidation.cpp`).
4. Client prototype: built an interactive CLI client to manually test `SEND` and `QUIT`.
5. Command extensions: added `LIST`, `READ`, and `DEL` handlers, following the same modular design and comment conventions.
6. Testing & cleanup: verified complete round-trip client/server communication, ensured protocol compliance, and unified code style and documentation.

Needed Adaptations

- Protocol adjustments: refined the format of responses to stay line-based and human-readable.
- File system organization: introduced per-user folders and incremental numeric message IDs.
- Robustness improvements: added error handling for socket interruptions (`EINTR`) and invalid user/message inputs.
- Consistency: standardized logging, function naming, and comment structure across all modules.
- Testing tooling: upgraded the client to fully exercise all commands interactively, enabling complete end-to-end testing.